

Segment-7

Template, Exception Handling and Standard Template Library

Templates in C++ Programming

Templates in C++ programming allows function or class to work on more than one data type at once without writing different codes for different data types.

For example, consider a situation where we have to sort a list of students according to their roll number and their percentage. Since, roll number is of integer type and percentage is of float type, we need to write separate sorting algorithm for this problem. But using template, we can define a generic data type for sorting which can be replaced later by integer and float data type

- Templates are often used in larger programs for the purpose of code re-usability and flexibility of program.
- The main advantage of using a template is the reuse of same algorithm for various data types, hence saving time from writing similar codes.
- The concept of templates can be used in two different ways:
 - Function Templates Or Generic functions
 - Class Templates Or Generic classes

Generic Function:

- A generic function defines a general set of operations that will be applied to various types of data.
- A generic function that represents several functions performing same task but on different data types
- A generic function is created using the keyword **template**.

The general form of a Generic function definition is shown here:

```
template < class T > return_type func_name ( parameter list )
{
// body of function
}
```

- T is a placeholder name for a data type used by the function and class is a keyword.
- We can also use keyword **typename** instead of **class**.
- When, an argument is passed to **some_function()**, compiler generates new version of **some_function()** to work on argument of that type.
- The templated type keyword specified can be either "**class**" or "**typename**": For example **template <typename T>** Both are valid and behave exactly the same.

For example, a function to add two integer and float numbers requires two functions. One function accept integer types and the other accept float types as parameters even though the functionality is the same. Using a function template, a single function can be used to perform both additions. It avoids unnecessary repetition of code for doing same task on various data types.

```

template <typename T>
T Sum(T n1, T n2)
{
    // Template function
    T rs;
    rs = n1 + n2;
    return rs;
}

int main()
{
    int A=10,B=20,C;
    long I=11,J=22,K;
    C = Sum(A,B);
    // Calling template function
    cout<<"\nThe sum of integer values : "<<C;
    K = Sum(I,J);
    // Calling template function
    cout<<"\nThe sum of long values : "<<K;
    return 0;
}

```

Output:

```

The sum of integer values : 30
The sum of long values : 33
Process returned 0 (0x0)   execution time : 0.052 s
Press any key to continue.

```

Why use Function Templates

- Templates are instantiated at compile-time with the source code.
- Templates are used less code than overloaded C++ functions.
- Templates are type safe.
- Templates allow user-defined specialization.
- Templates allow non-type parameters

Sample Question:

1. Assume you have a program that can Multiply several data types such as floating - point values, and integers. Each data types is served by a different function in the original program (Multiply for integer, Multiply for Doubles and Multiply for float). Your responsibility is to rewrite the code using the Multiply Generic Function. Every data types that supports multiplication should be able to be multiplied by the Multiply function. [Aut-23]
2. You are provided with a program that calculates the average of elements in an array for different data-types such integer, floating-point, double and characters etc. Each data types is served by a different function in the original program (average for integer, average for Doubles and average for float). Your task is to re-factor the code using a generic function called CalculateAverage(). The CalculateAverage() should be capable of calculating the average of any elements of any data types. [Sp-23]
3. Suppose you have a program that perform addition operations for different data types: integer, float, and strings. The original program provides separate functions for each data types (add for integer, add for doubles, add for strings). Your task is to re-factor the code using a generic function called Add. The Add functions should be capable of performing addition operation for any data types. [Sp-23]

Function Overloading vs Function Template

```
template <typename T>
T square(T x)
{
    T result;
    result = x * x;
    return result;
}

int main()
{
    int i, ii;
    double d, dd;
    i = 2;
    d = 2.2;
    ii = square(i);
    cout << "Square of Integer Number "<< " : " << ii << endl;
    dd = square(d);
    cout<< "Square of double number "<< " : " << dd << endl;
    return 0;
}
```

Output:

```
Square of Integer Number : 4
Square of double number : 4.84

Process returned 0 (0x0)   execution time : 0.048 s
Press any key to continue.
```

```
#include <iostream>
using namespace std;
int square (int x)
{
    return x * x;
}
double square (double x)
{
    return x * x;
}

int main()
{
    int i, ii;
    double d, dd;
    i = 2;
    d = 2.2;
    ii = square(i);
    cout << "Square of Integer Number "<< " : " << ii << endl;
    dd = square(d);
    cout<< "Square of double number "<< " : " << dd << endl;
    return 0;
}
```

```

Square of Integer Number : 4
Square of double number : 4.84

Process returned 0 (0x0)   execution time : 0.489 s
Press any key to continue.

```

Generic Classes:

- Like function templates, we can also create class templates for generic class operations.
- Sometimes, we need a class implementation that is same for all classes, only the data types used are different.
- Normally, we would need to create a different class for each data type OR create different member variables and functions within a single class.
- This will unnecessarily bloat our code base and will be hard to maintain, as a change in one class/function should be performed on all classes/functions.
- However, class templates make it easy to reuse the same code for all data types.
- The general form of a generic class declaration is shown here:

```

template <class T> class class_name
{
};

```

- To create a class template object, we need to define the data type inside a $\diamond$ when creation.

```

className<dataType> classObject;

```

Example:

```

template <class T>
class Calculator
{
private:
    T num1, num2;
public:
    Calculator(T n1, T n2)
    {
        num1 = n1;
        num2 = n2;
    }
    void displayResult()
    {
        cout << "Numbers are: " << num1 << " and " << num2 << "." << endl;
        cout << "Addition is: " << add() << endl;
        cout << "Subtraction is: " << subtract() << endl;
        cout << "Product is: " << multiply() << endl;
        cout << "Division is: " << divide() << endl;
    }
    T add() { return num1 + num2; }
    T subtract() { return num1 - num2; }
    T multiply() { return num1 * num2; }
    T divide() { return num1 / num2; }
};

```

```

int main()
{
    Calculator<int> intCalc(2, 1);
    Calculator<float> floatCalc(2.4, 1.2);

    cout << "Int results:" << endl;
    intCalc.displayResult();

    cout << endl << "Float results:" << endl;
    floatCalc.displayResult();

    return 0;
}

```

```

Int results:
Numbers are: 2 and 1.
Addition is: 3
Subtraction is: 1
Product is: 2
Division is: 2

Float results:
Numbers are: 2.4 and 1.2.
Addition is: 3.6
Subtraction is: 1.2
Product is: 2.88
Division is: 2

```

Exceptions Handling:

Exceptions are run time errors which occur at **runtime** like low disk space, division by zero, access to array outside its bounds. Exception Handling in C++ is a process to handle runtime errors or exceptions. We perform exception handling so that the normal flow of the application can be maintained even after runtime errors.

Exceptions are of two types. They are **synchronous** exceptions and **asynchronous** exceptions.

Errors like out of range index or overflow come under synchronous exceptions and errors which are beyond the program like keyboard interrupts, low disk space are called as asynchronous exceptions

C++ exception handling is built upon three keywords: **try**, **catch**, and **throw**.

throw – A program throws an exception when a problem shows up. This is done using a **throw** keyword.

catch – A program catches an exception with an exception handler at the place in a program where you want to handle the problem. The **catch** keyword indicates the catching of an exception.

try – A **try** block identifies a block of code for which particular exceptions will be activated. It's followed by one or more catch blocks.

In the most general terms, program statements that you want to monitor for exceptions are contained in a **try** block. If an exception occurs within the **try** block, it is thrown (using **throw**). The exception is caught, using **catch** in the catch block, and processed.

Example:

```
#include<bits/stdc++.h>
using namespace std;

int main()
{
    int num1,num2;
    try{
        cout<<"Enter the 1st Number:"<<endl;
        cin>>num1;
        cout<<"Enter the 1st Number:"<<endl;
        cin>>num2;
        if(num2==0){
            throw "Division by zero is not possible";
        }

        double result=(double)num1/num2;

        cout<<"Result="<<result;
    }

    catch( const char* x){
        cout<<x;
    }
}
```

Advantages of Exception Handling:

The benefits of exception handling are as follows,

- (a) Exception handling can control run time errors that occur in the program.
- (b) It can avoid abnormal termination of the program and also shows the behavior of program to users.
- (c) It can provide a facility to handle exceptions, throws message regarding exception and completes the execution of program by catching the exception
- (d) It can separate the error handling code and normal code by using try-catch block.
- (e) It can produce the normal execution flow for a program.
- (f) It can implement a clean way to propagate error. that is. when an invoking method cannot manage a particular situations, then it throws an exception and asks the invoking method to deal with such situation.

Re-throwing Exception

In the program execution, when an exception received by catch block is passed to another exception handler then such situation is referred to as re-throwing of exception.

This is done with the help of following statement,

throw;

The above statement does not contain any arguments. This statement throws the exception to next try catch block.

```
#include <iostream>

using namespace std;

// Function for Exception Thrown
void exceptionFunction() {
    // try block - inside Function
    try {
        //throw exception : In function
        throw 0;
    } catch (int i) {
        cout << "\nIn Function : Wrong Input :" << i;
        //re throw exception : out of the function
        throw;
    }
}

int main() {
    int var = 0;
    // try block - for exception code
    try {
        // Inside try block
        exceptionFunction();
    } // catch block
    catch (int ex) {
        // Code executed when exception caught
        cout << "\nIn Main : Wrong Input :" << ex;
    }
    return 0;
}
```

```
}
```

Output:

In Function : Wrong Input :0

In Main : Wrong Input :0

Sample Question:

1. What is exception handling? Describe the exception handling mechanism by using a suitable example.[Sp-24]
2. Describe the key components of c++ exception handling including try,catch,and throw with an example.[Aut-23]
3. What is an exception? What are the advantages of using exception handling mechanism in a program.[Sp-23]
4. Show the general form of try,catch and throw for exception handling.In your own words describe their operation[Aut-22]

Standard Template Library (STL)

The Standard Template Library (STL) is a set of C++ template classes to provide common programming data structures and functions such as lists, stacks, arrays, etc

STL has 4 components:

- **Algorithms**
- **Containers**
- **Functors**
- **Iterators**

Containers

The container can be described as objects that are used to store the collection of data. It helps in recreating and implementing complex data structures efficiently. The STL provides a range of containers, such as vector, list, map, set, and stack, which can be used to store and manipulate data.

1. Algorithms

The algorithm defines a collection of functions specially designed to be used on a range of elements. They act on containers and provide means for various operations for the contents of the containers.

Algorithms can be defined as functions applied to the containers and provide operation for the content of the container. for example : sort(), swap(), min(), max() ,find(), and binary_search() etc.

Iterators

Iterators are used to access data in the containers, and it helps in traversing through elements of containers using its functions. They can be incremented and decremented as per choice and help in the manipulation of data in the container.

Iterator functions are:

begin(): This function points the iterator to the first element of the container.

end(): This function points the iterator to the last element of the container.

Function Objects

A function object is also known as a functor; it is an object of a class that provides a definition for the operator(). Suppose you have declared an object of some class, then you can use that object just like the function.

Sample Question:

1. What is STL? Define container, iterator and algorithm with example [Aut-21]
2. Write a program to insert 20 items in STL vector, after that sort first N elements of that vector where N is an integer taken from keyboard. [Aut-22]
3. Write a program that initializes a vector with the values {5, 10, 2, 55, 60, 2}. Perform the following operations:
 1. Remove the element at index 2
 2. Insert the value at index 1
 3. Remove the last element from the vector [Sp-23, aut-23, Sp-24]

Tutorial Link for STL:

<https://www.youtube.com/watch?v=awY7oJ7wFhs&list=PLgLCjVh3O6Sgux985GYG22xkFt9z9Sq0>

OR

<https://www.youtube.com/watch?v=i2GHXA29NKw&list=PL0G2Ga9ALv6lgn-tZy5BxFzdr1AZ2E74l>

OR

<https://www.geeksforgeeks.org/the-c-standard-template-library-stl/>

Previous Question Answer

- c) Write a program that initializes a vector with the values {5, 10, 2, 55, 60, 3}. Perform the following operations:
1. Remove the element at index 2.
 2. Insert the value 15 at index 1.
 3. Remove the last element from the vector.

```
#include <iostream>
```

```
#include <vector>
```

```
Using namespace std;
```

```
int main() {
```

```
    // Initialize a vector
```

```
    vector<int> myVector = {5, 10, 2, 55, 60, 3};
```

```
    // Display the original vector
```

```
    cout << "Original Vector: ";
```

```
    for (int num : myVector) {
```

```
        cout << num << " ";
```

```
    }
```

```

cout << endl;

// 1) Remove the element at index 2
myVector.erase(myVector.begin() + 2);

// Display the vector after removing the element at index 2
cout << "After removing element at index 2: ";
for (int num : myVector) {
    cout << num << " ";
}
cout << endl;

// 2) Insert the value 15 at index 1
myVector.insert(myVector.begin() + 1, 15);
// Display the vector after inserting 15 at index 1
cout << "After inserting 15 at index 1: ";
for (int num : myVector) {
    cout << num << " ";
}
cout << endl;

// 3) Remove the last element from the vector
myVector.pop_back();

// Display the vector after removing the last element
cout << "After removing the last element: ";
for (int num : myVector) {
    cout << num << " ";
}
cout << endl;

return 0;
}

```

Muhammad Nazim Uddin
 Lecturer(Adjunct)
 Dept of CSE,IIUC